SAPIENTIA
ERDÉLYI MAGYAR
TUDOMÁNYEGYETEM

Sapientia Erdélyi Magyar Tudományegyetem

Marosvásárhelyi Kar

Számítástechnika szak

3D nyomtatási folyamat monitorizálás és hibadetektálás mesterséges intelligenciával

Készítette: Borbáth Mátyás-Levente

Témavezetők: Dr. Szántó Zoltán, Dr. Fehér Áron

2026

Kivonat

A 3D nyomtatási folyamat során különböző hibák léphetnek fel, amelyek anyagveszteséget, időveszteséget és akár a berendezés károsodását is okozhatják. A dolgozat célja egy olyan valós idejű monitorizáló rendszer fejlesztése, amely képfeldolgozási módszerekkel képes felismerni a nyomtatási hibákat. A javasolt megoldás egy többkamerás megfigyelőrendszerre épül, amely a nyomtatási folyamatot folyamatosan rögzíti, majd a képkockákat egy felhőalapú környezetben futó YOLOv8 neurális hálózat dolgozza fel. A modell egy saját annotált adathalmazon került betanításra, és 9 hibaosztály felismerésére képes, és 92.6% mAP50 pontossággal és 87.3% recall értékkel rendelkezik, illetve másodpercenként 3 képkockát dolgoz fel 23 ms inferencia idővel. A rendszer része egy értesítési mechanizmus is, amely anomália észlelése esetén azonnali visszajelzést ad a felhasználónak. A kísérleti eredmények alapján a rendszer alkalmas valós idejű hibadetektálásra, és hozzájárulhat a nyomtatási folyamat hatékonyságának növeléséhez, valamint az anyagveszteség csökkentéséhez.

Kulcsszavak: 3D nyomtatás,Konvoluciós Neuronháló, YOLOv8x, Rasberry Pi, Google Cloud, vertexAI, hiba, Terraform, Docker, Firebase

Tartalomjegyzék

1. Bevezető **1**

1.1. 3D nyomtatásról 1

1.1.1. Nyersanyagok 2

1.1.2. Felmerülő hibák és limitációk 5

2. Miért született meg a projekt? **14**

2.1. A projekt motivációja 14

2.1.1. Iparági relevancia és a nyomtatógazdálkodás kihívásai 15

2.1.2. A nyomtatási hibák műszaki és gazdasági hatása 16

2.1.3. Az automatizált monitorozás szükségessége 16

2.1.4. A projekt célkitűzései 17

3. Mélytanulási modellek **18**

3.1. Piackutatás és a megfelelő AI modell kiválasztása 18

3.1.1. Lehetséges eszközök 18

3.2. A projekt megvalósításához kiválasztott modell – YOLOv8 19

3.2.1. Bevezetés 19

3.2.2. Főbb jellemzők 20

3.2.3. Modellvariánsok 20

3.2.4. Telepítés és alapszintű használat 21

3.2.5. Saját adathalmaz felhasználása tanításhoz 21

3.2.6. Annotációs formátum 22

4. Hardver- és szoftverkörnyezet **23**
4.1. A teljes rendszer . . . 23
4.1.1. Kamera Agent-Rasberry pi . . . 24
4.2. Szolgáltatások a PI-n . . . 24
4.2.1. MediaMTX . . . 25
4.2.2. Képkockák kinyerése . . . 25
4.2.3. Cloudflare alagút . . . 27
4.3. Cloud Stack . . . 27
4.3.1. Terraform keretrendszer . . . 28
4.3.2. Terraform működése . . . 28
4.3.3. Google Cloud Projekt - GCP . . . 29
4.3.4. Vertex AI végpont . . . 30
4.3.5. Service Accounts . . . 30
4.3.6. Pub/Sub Topics . . . 31
4.3.7. Google Cloud Függvények . . . 32
4.3.8. Firestore Collections . . . 33
4.3.9. Konténer képek . . . 33
4.4. Dashboard . . . 34
4.5. Github Action Workflows . . . 35
4.5.1. Terraform lépések automatizálása . . . 35
4.5.2. Docker konténer felépítésének automatizálása . . . 36

5. Adatgyűjtés és előfeldolgozás **38**
5.1. Cím 1 . . . 38

6. Modell betanítása és validálása **39**
6.1. Cím 1 . . . 39

7. Tesztelés és kiértékelés **40**
7.1. Összefoglalás . . . 40

8. Rendszerintegráció és alkalmazás **41**

9. Összefoglalás és jövőbeli irányok **42**

10. Mellékletek **43**

Irodalomjegyzék **43**

A. Függelék **46**

A.1. Alfejezet 46

A.1.1. Cím 46

Ábrák jegyzéke

1.1. SLA nyomtató 2
1.2. FDM nyomtató 2
1.3. Döntési szempontok 2
1.4. Filament tekercsek 5
1.5. Szálazás a nyomtatás végén 6
1.6. Eltorzult darab 7
1.7. Elcsúszott rétegek 8
1.8. Teljes alul extrúdálás 9
1.9. Lokális alul extrúdálás 9
1.10. Felül extrúdált darab 10
1.11. Nyomtatás közben eldugult fúvóka és darab 11
1.12. Figyelmetlenségből a tárgyasztalon hagyott szerszám 12
1.13. Nem tapadó első réteg 12
1.14. Hibás nyomtatást követő spagetti halmaz 13

2.1. 3D nyomtató farm 15

4.1. A rendszer 24
4.2. MediaMTX stream pipeline 25
4.3. Terraform Lépések[1] 29
4.4. A `judge` konténer image életciklusa 34
4.5. A dashboard normál működési állapotban 35
4.6. A dashboard anomália detektálás esetén 35

4.7. A CI/CD pipeline lépései 37

1. fejezet

Bevezető

1.1. 3D nyomtatásról

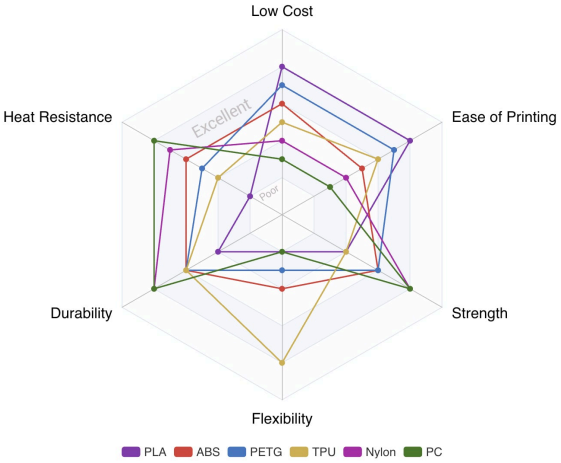
Mi is az additív gyártás, tömören azon folyamatokat hívjuk így melyek során egy darabot rétegről rétegre készítünk el, ellentéte a szubsztraktív gyártás, melynek során a darabot, egy szolid blokkból való elvéséssel állítjuk elő. A 3D nyomtatás maga egy addítív gyártási folyamat mely CAD vagy digitális 3D modellt épít fel különböző nyersagokat felhasználva, 3D nyomtatók segítségével. A folyamat maga egy 3D modellel kezdődik, melyet úgynevezett szeletelő programok segítségével, akár több száz vagy ezer egymással párhuzamos szeletre osztunk. A szoftver ezután a szeletelt modellt G-kóddá alakítja, mely utasítások sorozatát tartalmazza a 3D nyomtató számára. A 3D nyomtató ezek alapján építi fel majd a darabot. Ma a legelterjedtebb 3D nyomtató fajták az FDM (Fused Deposition Modelling) ezek a legismertebbek és a legelterjedtebbek a világon, működése a nyersanyagként használt műanyag megolvasztásán és annak vékony szállá extrúdáláson alapul. Egy kicsit komplexebb nyomtató fajta az SLA (Stereolithography) nyomtatók, ezek nyersanyagként speciális gyantát használnak, melyeket precízen irányított lézersugarak segítségével szilárdítanak meg. Ezen nyomtatók rendkívűli részletességre képesek, és nagyon precízek, viszont a darabokkal rengeteg utómunka van. Megemlíteném még A polyjet, binder jet, és a fémnyomtatásra használt, DMLS/SLM technológiákat. A dolgozat elkészítése során FDM nyomtatókon előforduló hibákat tanulmányoztam, innentől kezdve csak velük foglalkozunk.

1.1. ábra. SLA nyomtató[1]

1.2. ábra. FDM nyomtató[2]

1.1.1. Nyersanyagok

Az FDM nyomtatók nyersanyaga a filament, de felhasználási módtól függetlenül, több anyagot különböztetünk meg. Mindegyik anyagnak megvannak a maga tulajdonságai amik alapján kiválaszthatjuk a számunkra megfelelőt.



1.3. ábra. Döntési szempontok[3]

[1] *Fotó*:https://ro.pinterest.com/pin/338895940694784238/

[2] *Fotó*:https://eu.store.bambulab.com/products/a1?id=599117150694776844

PLA-Polylactic acid

A PLA legelterjedtebb filament a 3D nyomtatással foglalkozók körében. Ezen nyersanyag az egyik legkörnyezetbarátabb, hiszen megújuló, leggyakrabban kukorica keményítőből, cukornádból, tápiókából vagy más szénhidrátokban gazdag növényből állítják elő. Viszont nagyon fontos megjegyezni, hogy a közhiedelemmel ellentétben újrahasznosítása nem olyan egyszerű, csakis ipari körülmények között valósítható meg mivel magas hőt, páratartalmat és mikrobiológiai aktivítást igényel, ha ezen körülmények nem teljesülnek úgyanolyan káros mint bármilyen más műanyag a szeméthegyeken. PLA-val nyomtatni nem okoz nagy fejtörést, olvadáspontja alacsonyabb, mint más filamenteknek, mechanikai ereje a közepes kategóriába sorolható rugalmassága alacsony, színe a gyártási folyamatnak köszönhetően bármilyen lehet. Gyenge pontjai, a magas hőmérséklet és páratartalom és az UV sugárzásnak való kitettség.

ABS-Acrylonitrile butadiene styrene

Az ABS már a hobbiisták körében nem annyira elterjedt nyersanyag, mivel nyomtatása során, mérgező gázok keletkezhetnek, az ezek által okozott kár viszont megfelelő szellőztetéssel teljes mértékben elkerülhető. Az iparban nagyon szívesen nyomtatnak vele, hiszen az anyag tulajdonságai, mérsékelten jobbak mint a PLA-é, bár nyomtatása már nem olyan egyszerű, mert az anyag nagyon hajlamos a torzulása, ha ingadozik a hő a nyomtatótérben. Mechanikai keménysége magas és rugalmassága is jobb mint a pla-é. Az ABS-ből készült darabok időtállóak, nem kezdi ki őket sem a nedvesség, mintsem az uv sugárzás. Érdekesség képben kiemelném, hogy a legó kockák nagyrésze szintén ABS-ből készült.

PETG-Polyethylene Terephthalate Glycol

A PETG filament nagyszerű módon ötvözi az erőt flexibilitást, és a felhasználó barátságot, mondhatni ő az aranyközépút. Az anyag a műanyagüvegekhez is használt PET kis glycollal megfűszerezve, mely hatására kevésbé lesz törékeny és a minősége is javul. Mechanikai ereje igen magas és rugalmassága szintén közepes szintre sorolható. Az anyag rendkivűl ellenálló a nedvességgel szemben és kevésbé

[3] *Fotó*:https://all3dp.com/1/3d-printer-filament-types-3d-printing-3d-filament/

hajlamos a torzulásra. Szárítására különösen oda kell figyelni.

TPU-Thermoplastic Polyurethane

A TPU egy rendkivűl rugalmas, gumiszerű nyersanyag, mely a nyúlánkságáról keménységéről és becsapódás ellenálló képességőről híresült el. A szilárd filamentekkel szemben a TPU, hajlítható, nyújtható, összenyomható, és képess ütések elnyelésére. Mechanikai keménysége kicsit kisebb, viszont ezzel ellenben rugalmassága rendkívűl magas. Tökéletes olyan darabok nyomtatására melyek folyamatos használat alatt állnak majd, például szíjak, abroncsok és társaik. Az anyag hajlamos a szálazásra, nyúlánksága miatt, így sokkal lassabban kell nyomtatni mint a szilárd társait, a nyomatót nagyon gyakran hozzá kell majd kalibrálni.

Nylon-Polyamide

A Nylon, mint nyersanyag főként iparban használatos, nagyszerű ereje és tartóssága miatt gyakran a biztos választás. A nylon nyomtatása a fúvókák, és a zárt nyomtató terek fejlesztésével vált elérhetővé, melyekben nagyobb kontrol van a nedvesség tartalom és a hőmérséklet fölött. Mechanikai keménysége elsőosztályú, rugalmassága kiváló, tartóssága pedig messze felülmúlja társaiét.

PC- Polycarbonate

A policarbonate amellet, hogy a mai napig a világon a legkeményebb és legtartósabb filament címét tudhatja magáénak, rendkivűl ellenálló mint a hőnek mint a nedvességnek. Nyomtatása döcögős hobbiisták számára ugyanis nagyon sok hőt igényel, a zökkenőmentes nyomtatáshoz a fúvókának a 260-310°C tartományban kell lennie, és a tárgyasztalt zár térbe kell telepíteni, ezzel is gátolva a hőingadozást.

Egyébb nyersanyagok

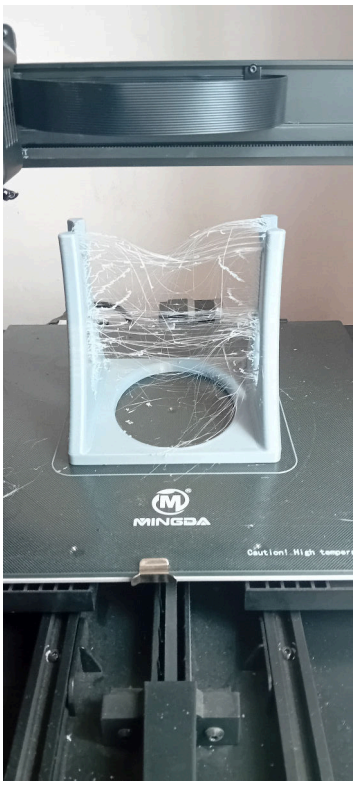
A fent említettek mellett, persze ma már mégtöbb filament áll rendélkezésünkre, viszont ezek a dolgozat szempontjából nem annyira relevánsak, mivel a fent tárgyalt variánsok keverékei. Ilyen például, az ún selyem PLA, melynek mechanikai keménysége alacsonyabb, viszont szebb, fényesebb textúrával rendelkezik, tökéletes Olyan darabok nyomtatására, melyeknél az esztétika fontosabb mint a mechanikai felhasználás, például a dísztárgyak. Az Wood-PLA, egy olyan PLA variáns mely fa darabkákkal ötvözött, a nyomtatás után a darab fás tapintásnak örvend majd, ebből létezik fémmel ötvözött variáns is.

1.4. ábra. Filament tekercsek

1.1.2. Felmerülő hibák és limitációk

A 3D nyomtatási folyamat során rengeteg hiba felmerülhet. mely több okból is keletkezhet, a nedves anyagtól, a rossz kallibrációig, vagy akár komoly hardver hibáig is. Beszéljünk egy kicsit ezen hibákról.

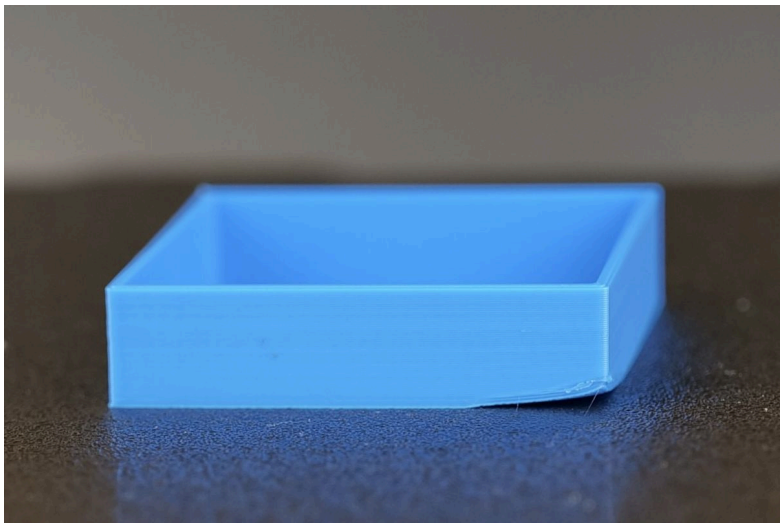
Stringing- Avagy szálazás



1.5. ábra. Szálazás a nyomtatás végén

Ezen hiba fő jellegzetessége apró szinte áttetsző szálak megjelenése a darab élei között, ennek oka hogy a fúvóka utazás közben olvadt műanyagot húz a levegőben amíg A pontból B pontba ér. Legfőbb oka általában nem megfelelően kiszárított filament használata a nyomtatáshoz. Ezen szálak utómunkával eltávolíthatóak, de nem garantált, hogy nyomuk teljesen eltávolítható. A hibát még okozhatja, nem megfelelő visszahúzás érték, túl forró fúvóka, és mocskos fúvóka is.

Warping- Avagy eltorzulás

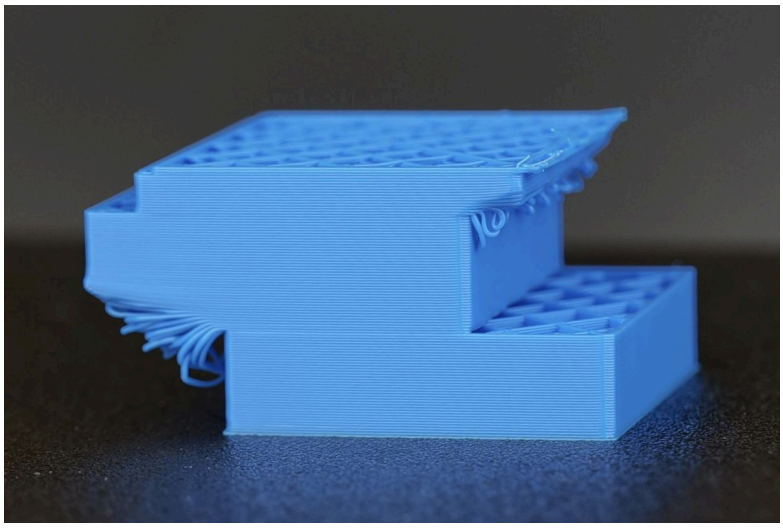


1.6. ábra. Eltorzult darab[4]

Eltorzulás legtöbbször akkor jelentkezik, amikor egy darab alján éles szögek találhatóak, például négyzet, vagy egyébb sokszög. Fontos beavatkozó viszont a tárgyasztal hője, tisztasága is a megfelelő tapadás fenttartásához. A hiba maga kiküszöbölhető egy úgynevezett skirt, avagy szoknya alkalmazásával a szeletelőben, mely búnusz tapadást biztosít a sarkakon.

[4] *Fotó*:https://wiki.bambulab.com/en/knowledge-sharing/printed-model-warping

Layer shift-Avagy réteg eltolódás



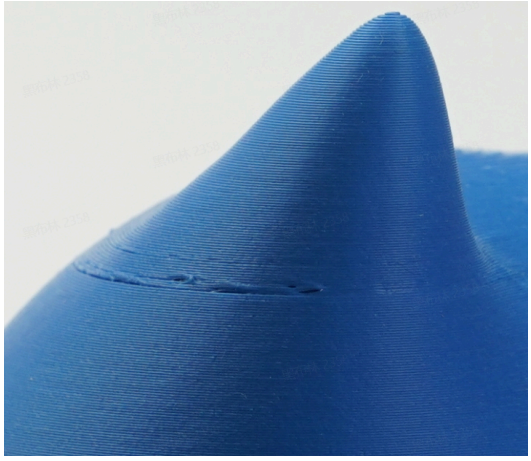
1.7. ábra. Elcsúszott rétegek[5]

A nyomtatott rétegek eltolódása végzetes hiba a darab számára. Főként akkor jelenik meg amikor eltérés van a G-kódban található koordítnáta és a fúvóka világi koordínátái között. Három fő oka van, túl nagy nyomtatási sebesség, nem megfelelő Z-offset mely miatt a fúvóka beleakadhat a darabba ezzel koordináta különbséget előídézve, vagy a nyomtatót ért fizikai trauma, esetleg laza sínek vagy merevítők.

[5] *Fotó*:https://wiki.bambulab.com/en/knowledge-sharing/layer-shifts

Under extrusion- Avagy alul extrúdálás

1.8. ábra. Teljes alul extrúdálás[6]



1.9. ábra. Lokális alul extrúdálás[7]

Ezen hibának két külön fajtáját különböztetjük meg, a teljes illetve a lokális alul extrúdálást. Ezen hiba okai lehetnek mind hardveres, de előokozhatja nem megfelelő szeletelés is. Főbb előidéző okok közé tartozik, hogyha, eldugul a ptfe cső, vagy idegen test kerül bele, beakadt a filament tekercs és nem jút elég a fúvókához, fogaskerék probléma, vagy részleges dugúlás a fúvókában vagy a hotendben. Amennyiben a problémát szoftveres ok váltja ki a Maximum Volumetric speed illetve Flow ratio nevű beállításokkal van a probléma. A mai modern nyomtatók már fel vannak szerelve autómatikus adagolás kallibrációval, ezzel segítve a felhasználót.

[6] *Fotó*:https://wiki.bambulab.com/en/filament-acc/filament/print-quality/under-extrusion
[7] *Fotó*:https://wiki.bambulab.com/en/filament-acc/filament/print-quality/under-extrusion

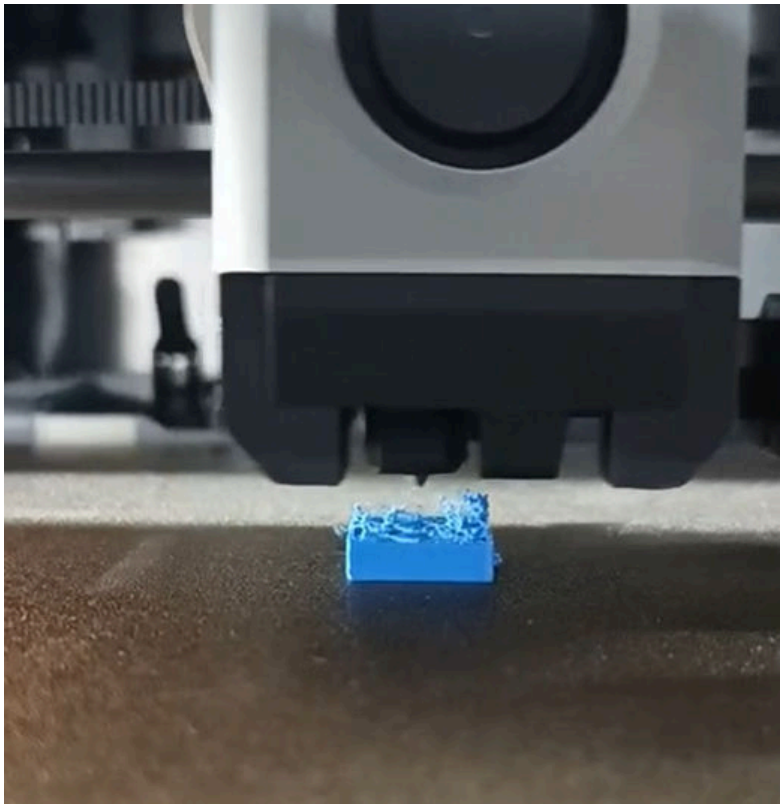
Over extrusion- Avagy felül extrúdálás

1.10. ábra. Felül extrúdált darab[8]

Ugyanazon probléma mint az alul extrúdálás, egy annyi csavarral, hogy itt nem túl kevés anyag jut a fúvókához hanem pont, hogy túl sok.

[8] *Fotó*:https://www.3dsourced.com/guides/3d-print-over-extrusion/

Nozzle Clog- Avagy fúvóka dugulás



1.11. ábra. Nyomtatás közben eldugult fúvóka és darab[9]

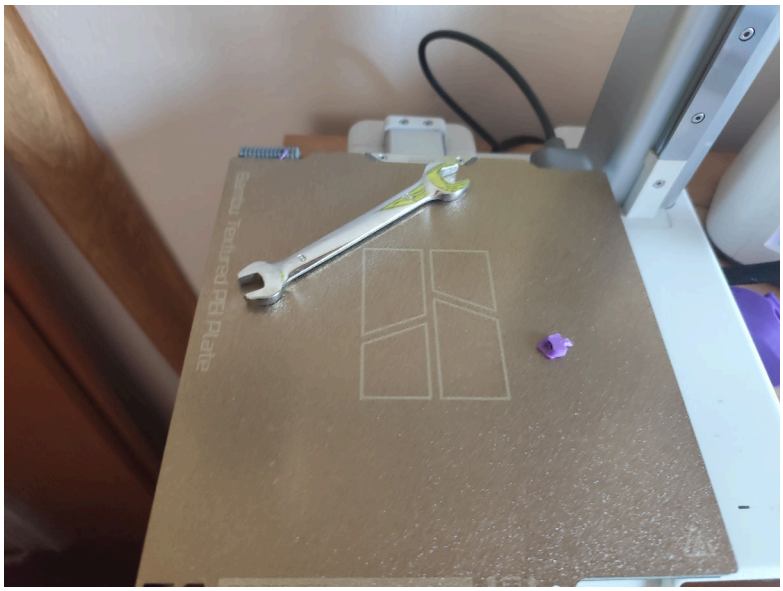
Egy sajnos igen gyakori probléma mely akár a nyomtatónk "életébe" is kerülhet. Legfőbb oka, hogy a filament amikor a ptfe csövön keresztül a forró kamrába, majd pedig a fúvókába jut nem teljesen tiszta, kis por vagy egyébb szennyező anyagok is bekerülnek, melyek a fúvóka dugulását okozhatják. Amennyiben nem vesszük észre hamar az olvadt műanyag felgyülhet a fúvókában és belülről szétfeszíti azt, ezzel jelentős anyagi kárt okozva.

Foreign object on the printbed- Avagy idegen tárgy a tárgyasztaklon

Ezen hiba se nem hardveres se nem szoftveres hanem szimplán a kezelő figyelmetlensége miatt következik be, akik a tapasztalatomból legtöbbször olyan izgatott a szeletelést követően, hogy elfelejti ellenőrizni, hogy leürítette-e a tárgyasztalt, vagy csak karbantartás után nem pakolt el maga után.

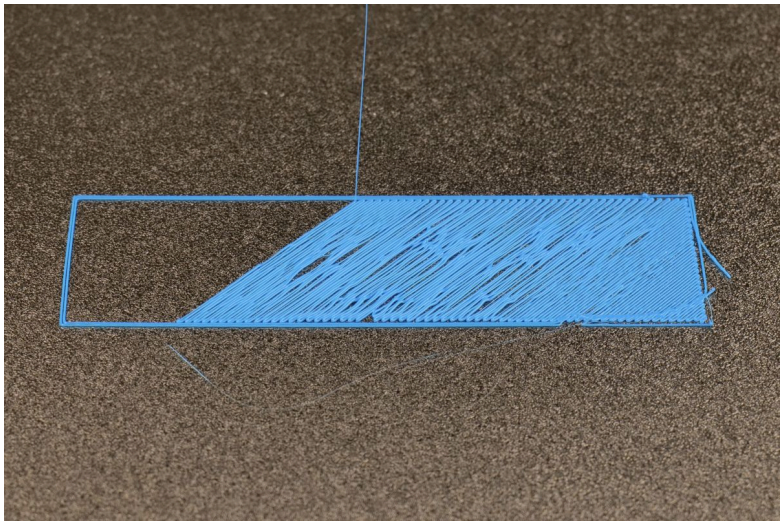
[9] *Fotó*:https://wiki.bambulab.com/en/filament-acc/filament/print-quality/clog

Az idegen tárgyak a fúvókában és a fejben súlyos anyagi kárt okozhatnak viszont, hiszen a legtöbb nyomtató nem képes detektálni, ha a tárgyasztal üres vagy sem.



1.12. ábra. Figyelmetlenségből a tárgyasztalon hagyott szerszám

First layer not sticking- Avagy az első réteg nem tapad



1.13. ábra. Nem tapadó első réteg[10]

[10] *Fotó*:https://wiki.bambulab.com/en/knowledge-sharing/first-layer-not-sticking

A 3D nyomtatási folyamat során a legfontosabb mozzanat az első réteg, ha ez nem tapad oda akkor az egész darab kuka, fontos figyelemmel kísérni, amennyiben észrevétlen marad, az Spagettihez, és rengeteg anyag veszteséggel jár. Főbb okai, ha a tárgyasztal nem megfelelő tisztaságú, rossz fúvóka tárgyasztal távolság (Z-offset), vagy rossz asztal szintezés. Könnnyen elkerülhető a megfelelő beállításokkal és a tárgyasztal karban-és tisztán tartásával.

Spagetti- Avagy az anyag a levegőbe adagolódik

Rengeteg oka lehet, de a legfőbb előokozója, ha az első réteg nyomtatási folyamata sikertelennek bizonyult, a fúvókát ez nem érdekli, ő követi a G-kód által megadott utasításokat és addig nyomja a levegőbe az anyagot, ameddig a végére nem ér, vagy a kezelő azt meg nem állítja. Fontos kiemelni, hogy a tapadás elvesztése a nyomtatási folyamat bármely pillanatában bekövetkezhet, amennyiben nem megfelelő support beállításokat alkamaztunk a szeletelőben, vagy éppen a supportok engedik el a tárgyasztalt, ezzel védtelenül hagyva a darabot.

1.14. ábra. Hibás nyomtatást követő spagetti halmaz

2. fejezet

Miért született meg a projekt?

2.1. A projekt motivációja

Az additív gyártás – ezen belül is az FDM (Fused Deposition Modelling) alapú 3D nyomtatás – az elmúlt évtizedben kiemelkedő szerepet töltött be a modern gyártóiparban. A technológia lehetővé teszi egyedi geometriák, kis sorozatú alkatrészek és funkcionális prototípusok gyors, gazdaságos előállítását, melyre a hagyományos szubsztraktív eljárások nem képesek ilyen rugalmassággal. Ezen előnyök eredményeképpen a 3D nyomtatás napjainkra az ipari termelésen túl az egészségügytől az űrtechnológiáig számos területen meghonosodott.

Ugyanakkor a technológia széleskörű elterjedésével párhuzamosan az üzemeltetési kihívások is egyre összetettebbekké válnak. A nyomtatási folyamatok – különösen FDM technológia esetén – számos tipikus meghibásodásra hajlamosak, amelyek anyagveszteséget, időveszteséget, illetve súlyosabb esetben a berendezés megrongálódását okozhatják. Ilyen körülmények között a hibák kézi, szemrevételezésen alapuló felismerése nem csupán lassú és körülményes, hanem nagyüzemi környezetben gyakorlatilag kivitelezhetetlen.

Jelen fejezet ismerteti azokat az iparági és műszaki tényezőket, amelyek indokolttá teszik egy valós idejű, mesterséges intelligencián alapuló hibadetektor rendszer kifejlesztését, majd összefoglalja a projekt konkrét célkitűzéseit.

2.1.1. Iparági relevancia és a nyomtatógazdálkodás kihívásai

A 3D nyomtatás iparági térhódítása az elmúlt évtizedben látványos volt. A gyártóüzemekben egyre elterjedtebbé válnak az úgynevezett nyomtatófarmok, ahol egyetlen helyszínen tíz vagy akár száz nyomtatóegység üzemel egyidejűleg [1]. Ilyen léptékű infrastruktúra esetén a manuális felügyelet – azaz a kezelő személyes jelenlétével megvalósuló ellenőrzés – nem valósítható meg hatékonyan: a karbantartó személyzet sem az egyes gépek állapotát, sem az éppen zajló nyomtatásokat nem képes folyamatosan nyomon követni.

2.1. ábra. 3D nyomtató farm [1]

A farm-üzemeltetés sajátos kihívást jelent a hibadetektálás szempontjából is. Ha egyetlen gépen bekövetkező hiba – például szálazás, réteg-eltolódás vagy tapadásvesztés – észrevétlen marad, az anyag- és időveszteség sokszorozódhat: a nyomtató akár órákon át termel selejtet, mire a kezelő beavatkozik. Súlyosabb esetben a fúvóka károsodása vagy a tárgyasztal megsérülése komoly javítási

költséget von maga után. Az automatizált monitorozás ebben a kontextusban nem csupán kényelmi eszköz, hanem gazdasági szükségszerűség [2].

2.1.2. A nyomtatási hibák műszaki és gazdasági hatása

A 3D nyomtatási folyamat több okból is hibásodhat meg: a nem megfelelően kondicionált filamenttől és hibás szeletelési beállításoktól egészen a kalibrációs eltérésekig és hardveres meghibásodásokig. A leggyakoribb és legsúlyosabb következményekkel járó hibatípusok a következők:

- **Tapadásvesztés és spagetti:** Az első réteg elválása a tárgyasztaltól az egyik legelterjedtebb és legnehezebben megelőzhető hiba. Ha a darab elveszíti rögzítettségét, a fúvóka a levegőbe extrudálja az anyagot, egy jellegzetes, rendezetlenül összegubancolódott szálkupacot – köznyelvben „spagettit" – hozva létre. Az anyagveszteség ilyenkor akár több tíz gramm is lehet.
- **Réteg-eltolódás (layer shift):** A koordináta-rendszerbeli eltérés miatt a nyomtatott rétegek elcsúsznak egymáshoz képest, a darab szerkezete meggyengül vagy teljesen használhatatlanná válik.
- **Alul- és felülextrúdálás:** Az anyagadagolás egyensúlytalansága a darab struktúrális integritását veszélyezteti; szélső esetben fúvókaduguláshoz vezet.
- **Szálazás (stringing):** Bár nem feltétlenül teszi funkcióképtelenné a darabot, az esztétikai és precíziós igényekre érzékeny alkalmazásokban elfogadhatatlan minőséget eredményez.

Ezek a hibák nemcsak az egyes darabok minőségét rontják le, hanem – ha felügyelet nélkül maradnak – a gyártóeszköz élettartamát is csökkenthetik. A nyomtatófarmok üzemeltetői számára különösen fontos a korai, automatikus jelzés, hiszen a kiesési idő és az anyagköltség közvetlen üzleti veszteséget jelent.

2.1.3. Az automatizált monitorozás szükségessége

Az automatizált képfeldolgozás és a mélytanulási módszerek rohamos fejlődése napjainkra megteremtette az alapot a valós idejű ipari hibadetekcióhoz. A konvolúciós neurális hálózatok (CNN-ek)

képesek vizuális mintázatok felismerésére akkor is, ha azok az emberi szem számára nehezen, vagy csak figyelmes vizsgálattal azonosíthatók. Az Obico (korábban The Spaghetti Detective) projekt például egy nyílt forráskódú, mesterséges intelligencián alapuló megoldást nyújt az FDM nyomtatók felügyeletéhez, és igazolta, hogy ilyen rendszerek a valóságban is megbízhatóan működtethetők [2].

Ugyanakkor az elérhető megoldások jellemzően egyetlen szögből figyelik a nyomtatót, ami korlátozza az észlelhető hibatípusok körét. Egy, a tárgyasztalt több kameraállásból felügyelő rendszer lényegesen pontosabb képet ad a folyamatról, és olyan hibák is detektálhatóvá válnak, amelyek egyetlen nézőpontból nem látszanak.

2.1.4. A projekt célkitűzései

A fentiekben ismertetett iparági igények és műszaki lehetőségek alapján a jelen projekt célja egy olyan skálázható, valós idejű monitorozó- és hibadetektor rendszer megtervezése és megvalósítása, amely:

- Minden felügyelt nyomtatót egyszerre, **több kameraállásból** figyel;
- **Konvolúciós neurális hálózat** segítségével automatikusan azonosítja a leggyakoribb FDM hibatípusokat;
- A detektált hibát **kategorizálja** – megkülönböztetve a nyomtatás közbeni beavatkozást igénylő, illetve az azonnali leállítást indokló eseteket;
- A felhasználót **valós időben értesíti** a szükséges teendőkről;
- **Skálázható** architektúrán alapul, hogy nyomtatófarmok esetén is alkalmazható legyen.

A rendszer megvalósításával jelentős anyag-, idő- és költségmegtakarítás érhető el, mivel a hibák korai felismerése megakadályozza a selejtes darabok keletkezésének elhúzódását és a berendezések esetleges sérülését.

3. fejezet

Mélytanulási modellek

3.1. Piackutatás és a megfelelő AI modell kiválasztása

3.1.1. Lehetséges eszközök

Képfelismerési feladatok megvalósítására számos gépi tanulási keretrendszer áll rendelkezésre, így a projekt megvalósítása során nem volt szükség egy teljes neurális háló felépítésére a semmiből. A megfelelő eszköz kiválasztásánál több szempontot is figyelembe kellett venni: a modell pontosságát, valós idejű felhasználhatóságát, integrálhatóságát, valamint a rendelkezésre álló hardver- és szoftverkörnyezettel való kompatibilitást. Kiemelt szempont volt, hogy a betanított modell könnyen illeszthető legyen egy Jupyter Notebook alapú, majd Google Cloud infrastruktúrára épülő pipeline-ba, ezzel csökkentve a tanítás és futtatás során felmerülő hardveres korlátokat.

Faster R-CNN

A Faster R-CNN (*Region-based Convolutional Neural Network*) egy kétfázisú objektumdetekciós neurális hálózat, amelyet 2015-ben a Microsoft Research kutatói publikáltak [3]. Működése három egymást követő lépésre bontható. Az első fázisban egy előre betanított konvolúciós hálózat vizuális jellemzőtérképet (*feature map*) készít a bemeneti képből. Ezt követi a *Region Proposal Network* (RPN), amely a jellemzőtérkép alapján jelöltterületeket generál, és lényegesen gyorsabb a korábbi, manuális régiókereső algoritmusoknál. A harmadik lépésben az RoI (*Region of Interest*) pooling egy-

séges méretre skálázza a javasolt régiókat, majd egy másodlagos osztályozó hálózat minden régiót külön kiértékel, meghatározza az osztálycímkéket, és finomítja a határoló dobozok koordinátáit.

A Faster R-CNN előnye a kétfázisú feldolgozásból fakadó magas detekciós pontosság, és az előre betanított konvolúciós gerinc könnyen cserélhető más hálózatokra. Hátrányaként az alacsony inferenciasebesség emelhető ki, ami kizárja a valós idejű alkalmazásokban való felhasználást. Emellett a modell mérete és erőforrásigénye lényegesen nagyobb, mint az egyfázisú detektoroké, és a Jupyter Notebook-alapú pipeline-ba való integrálása is jóval összetettebb.

YOLOv5

A YOLOv5 (*You Only Look Once*, 5. verzió) egy egyfázisú objektumdetekciós neurális hálózat, amelyet az Ultralytics cég adott ki 2020-ban [4]. Az iparban széleskörűen elterjedt, mivel könnyen betanítható, jól dokumentált, és minimális konfigurációval felhasználható. Az architektúra három fő részből áll: a *gerinc* (*backbone*) a Cross Stage Partial Darknet segítségével a képből kinyeri a vizuális jellemzőket; a *nyak* (*neck*) összevonja a különböző méretarányú jellemzőket, biztosítva a többméretű objektumdetekciót; a *fej* (*head*) horgonydobozok (*anchor boxes*) alapján három különböző felbontásban állítja elő a detekciós kimenetet.

A YOLOv5 alacsony inferenciaideje miatt alkalmas valós idejű felismerésre, a hiperparaméterek és augmentációk előre konfigurálva vannak, és a veszteségfüggvény is jól hangolt. Bár a projekt szempontjából megfelelő jelölt lett volna, az Ultralytics által 2023-ban kiadott YOLOv8 mind architektúra, mind teljesítmény tekintetében felülmúlja elődjét, ezért a választás arra esett.

3.2. A projekt megvalósításához kiválasztott modell – YOLOv8

3.2.1. Bevezetés

A YOLOv8 (*You Only Look Once*, 8. verzió) az Ultralytics cég által 2023-ban kiadott objektumdetekciós modellcsalád [5]. A valós idejű objektumdetektálásban nyújtott kiemelkedő teljesítménye, valamint a könnyű integrálhatósága indokolta a választást, mivel a 3D nyomtatási folyamat élő kamerabemenetről történő felügyeletéhez és a hibák automatikus azonosításához mindkét tulajdonság

elengedhetetlen.

3.2.2. Főbb jellemzők

A YOLOv8 keretrendszer esetében az Ultralytics teljes mértékben újragondolta a modell architektúráját. Az előző verzióhoz képest egy saját fejlesztésű keretrendszerre tért át, amelynek köszönhetően a modell gyorsabbá és stabilabbá vált, ugyanakkor ez visszafelé inkompatibilitást jelent a YOLOv5-tel szemben [5]. A keretrendszer az objektumdetekción kívül szegmentálást, pozícióbecslést, osztályozást és objektumkövetést is támogat. A tanítási és detekciós folyamat segédszkriptek nélkül, API-hívásokon vagy parancssori felületen (CLI) keresztül is vezérelhető. A betanított modell exportálható többek között ONNX, CoreML és TensorRT formátumokba is.

3.2.3. Modellvariánsok

A YOLOv8 több méretű változatban érhető el, amelyek különböző felhasználási célokhoz és hardverkörnyezethez igazíthatók [5].

3.1. táblázat. YOLOv8 modellvariánsok

Modell	**Méret**	**Leírás**
`yolov8n`	Nano	A legkisebb variáns, rendkívül gyors és minimális hardverigényű.
`yolov8s`	Small	Kis méretű modell, gyors inferenciával, alacsony erőforrásigénnyel.
`yolov8m`	Medium	Kiegyensúlyozott kompromisszum sebesség és pontosság között.
`yolov8l`	Large	Nagy pontosságú modell, erősebb GPU-t igényel.
`yolov8x`	Extra Large	A legnagyobb pontosságot nyújtó variáns, egyúttal a legtöbb erőforrást igénylő.

3.2.4. Telepítés és alapszintű használat

A csomag a Python csomagkezelőn keresztül telepíthető [5]:

```
pip install ultralytics
```

Egyszerű képfeldolgozási példa:

```
from ultralytics import YOLO
model = YOLO("yolov8n.pt")
results = model("stringing.jpg")
results.show()
```

3.2.5. Saját adathalmaz felhasználása tanításhoz

A YOLOv8 modell saját adathalmazon is betanítható, amennyiben a képek megfelelően fel vannak annotálva és az exportált annotációk formátuma YOLO-kompatibilis [5]. Ehhez egy `data.yaml` konfigurációs fájlra van szükség, amely megadja a tanítási és validációs adathalmazok elérési útjait, az osztályok számát és elnevezéseit.

A `data.yaml` konfigurációs fájl szerkezete:

```
train: images/train
val: images/val
nc: 8
names: ['stringing', 'warping', 'layer_shift', 'under_extrusion',
        'over_extrusion', 'nozzle_clog', 'foreign_object_on_print_area',
        'not_sticking', 'spagetti']
```

Tanítási folyamat indítása CLI-n keresztül:

```
yolo task=detect mode=train model=yolov8n.pt data=data.yaml epochs=50 imgsz=640
```

3.2.6. Annotációs formátum

YOLO-formátumban minden képfájl mellé egy azonos nevű `.txt` fájl tartozik. Minden sor egy objektumot ír le; egyetlen fájl több objektum leírását is tartalmazhatja:

```
<class_id> <x_center> <y_center> <width> <height>
```

Az értékek lebegőpontos számok, a kép méretéhez arányosan 0 és 1 közé normalizálva. Az annotáláshoz a CVAT (*Computer Vision Annotation Tool*) szoftvert [6] alkalmaztuk.

3.2. táblázat. YOLOv8 előnyei és hátrányai a projekt szempontjából

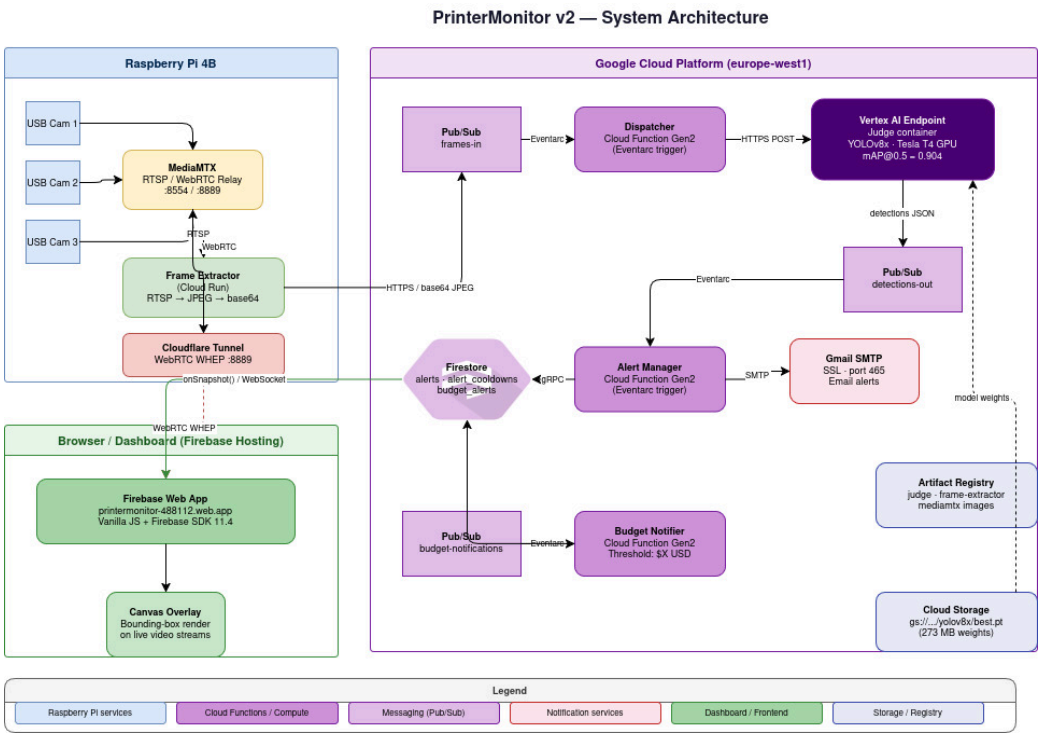
Előnyök	**Hátrányok**
Valós idejű objektumdetektálás	Nagy modellek (pl. YOLOv8x) jelentős GPU-erőforrást igényelnek
Egyszerű és megismételhető tanítási folyamat	Kis adathalmazon hajlamos a túltanulásra
Kis modellek is magas pontosságot érnek el	Zsúfolt vagy részben takart objektumok detektálása nehézkes
Docker-konténerizációs támogatás	Valós idejű feldolgozás teljesítménye hálózati késleltetéstől függhet
Aktív fejlesztői közösség és folyamatos frissítések	Egyedi osztályokhoz saját adathalmaz gyűjtése szükséges

4. fejezet

Hardver- és szoftverkörnyezet

4.1. A teljes rendszer

A rendszer egy edge–cloud hibrid architektúrára épül, amelynek két fő pillére van: egy helyszínen elhelyezett Raspberry Pi edge eszköz, valamint a Google Cloud Platform felhőinfrastruktúrája. A Pi feladata a kamerás képrögzítés és az adatok előfeldolgozása, míg a felhő végzi a gépi tanulás alapú hibadetekciót, az adattárolást és a felhasználói értesítéseket. Az adatfolyam iránya: Raspberry Pi → Pub/Sub (`frames-in`) → Dispatcher Cloud Function → Vertex AI végpont (YOLO inferencia) → Pub/Sub (`detections-out`) → Alert-Manager Cloud Function → Firestore + e-mail értesítés → Dashboard.



4.1. ábra. A rendszer

4.1.1. Kamera Agent-Rasberry pi

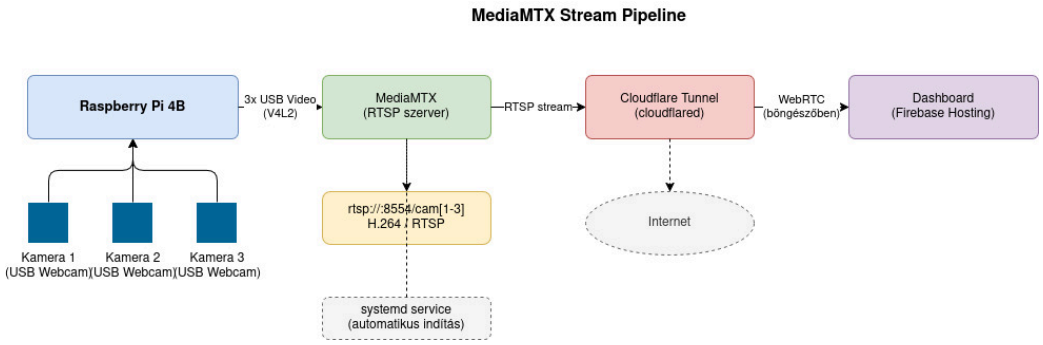
A kamera képek előállításért felelő kamera Agent szerepét egy rasberry pi 4b 8gb tölti be,[7] 3 darab rácsatolt webkamerával kiegészítve. A rasberry pi mint edge eszköz egy elég verszatilis fegyver a kezünkben. Keveset is fogyaszt, megfelelő számú usb porttal is rendelkezik, és az erőforrások is megfelelőek számunkra. A rasberry Os, pedig tökéletes a szükséges docker konténerek és python scriptek futtatására.

4.2. Szolgáltatások a PI-n

A pi-n három szolgáltatás fut, melyek az indítás után azonnal elindulnak systemd szolgáltatásként. Ezek nélkül a rendszer nem valósulhatna meg.

4.2.1. MediaMTX

A MediaMTX (korábban rtsp-simple-server) egy könnyűsúlyú, nyílt forráskódú médiaközvetítő szerver, mely Go programozási nyelven íródott.[8] Szerepe a rendszerben, hogy a webkamerák által előállított nyers videófolyamokat RTSP protokoll szerint tegye elérhetővé a hálózaton. A pi-n systemd szolgáltatásként fut, így az eszköz indításakor automatikusan elindul, emberi beavatkozás nélkül. Konfigurációs fájlja definiálja a három kamerastream elérési útját, a kódolási paramétereket, valamint az újracsatlakozási logikát. A H264-es kódolású RTSP végpontokat a Cloudflare alagúton keresztül éri el a dashboard, ahol WebRTC formátumban kerülnek megjelenítésre.



4.2. ábra. MediaMTX stream pipeline

4.2.2. Képkockák kinyerése

A képkockák kinyerésért a `frame-extractor` szolgáltatás felelős, melynek belépési pontja a `main.py` Python script. Feladata, hogy a három RTSP streamből folyamatosan képkockákat olvasson ki, azokat feldolgozza, majd JSON üzenetként publikálja a `frames-in` Pub/Sub csatornára, ahonnan a `dispatcher` Cloud Függvény továbbítja őket a Vertex AI végpontra. A szolgáltatás a pi-n systemd szolgáltatásként fut, és az `sa-frame-extractor` service account kulcsával hitelesíti magát a Google Cloud felé.

Szálkezelés és capture loop

A script induláskor beolvassa a kamerák RTSP URL-jeit és azonosítóit a környezeti változókból, majd minden egyes kamerához egy dedikált szálat (`threading.Thread`) indít. Minden szál egy önálló `capture_loop` függvényt futtat, amely az OpenCV[9] `VideoCapture` osztályán keresztül csatlako-

zik az adott RTSP streamhez. Ha a kapcsolat megszakad — például a kamera újraindul vagy a hálózat elvész —, a loop automatikusan újracsatlakozást kísérel meg 5 másodpercenként, így a rendszer emberi beavatkozás nélkül helyreáll.

Képfeldolgozás és publikálás

Sikeres kapcsolat esetén a loop `CAPTURE_FPS` (alapértelmezetten 2 fps) sebességgel olvas képkockákat. Minden egyes képkockán a következő feldolgozási lépések hajtódnak végre:

1. Az OpenCV a kamera natív felbontásáról átméretezi a képet a konfigurált `FRAME_WIDTH` × `FRAME_HEIGHT` méretre (alapértelmezetten 640×480 pixel).
2. A képkocka JPEG formátumba kerül kódolásra a beállított `JPEG_QUALITY` értékkel (alapértelmezetten 70), csökkentve az átviteli méretet.
3. A JPEG bájttömb Base64 kódolással ASCII szöveggé alakul, hogy JSON üzenetbe ágyazható legyen.
4. A script egy JSON objektumot állít össze, amely tartalmazza a `camera_id` azonosítót, a `seq` szekvenciaszámot, az `ts` UTC-időbélyeget, valamint a `data_b64` mezőt a Base64-kódolt képpel.
5. A kész JSON payload publikálásra kerül a Pub/Sub `frames-in` topicra.

Health check szerver

Mivel a szolgáltatás Cloud Run-kompatibilis konténerként is üzemeltethető, a script egy egyszerű HTTP szervert indít a `PORT` környezeti változóban megadott porton (alapértelmezetten 8080). Ez a szerver minden `GET` kérésre `200 OK` választ ad, lehetővé téve a platform számára, hogy ellenőrizze a konténer élettartamát.

Konfiguráció

A script teljes mértékben környezeti változókon keresztül konfigurálható, nem tartalmaz beégetett értékeket. A legfontosabb paramétereket az alábbi táblázat foglalja össze:

Változó	Alapértelmezett	Leírás
`GCP_PROJECT`	— (kötelező)	Google Cloud projekt azonosítója
`FRAMES_TOPIC`	— (kötelező)	Pub/Sub topic neve
`RTSP_URLS`	— (kötelező)	Vesszővel elválasztott RTSP URL-ek
`CAMERA_IDS`	cam1,cam2,cam3	Kamera azonosítók
`CAPTURE_FPS`	2	Másodpercenkénti képkockák száma
`JPEG_QUALITY`	70	JPEG tömörítési minőség (0–100)
`FRAME_WIDTH`	640	Képkocka szélessége pixelben
`FRAME_HEIGHT`	480	Képkocka magassága pixelben
`PORT`	8080	Health check HTTP szerver portja

4.1. táblázat. A `frame-extractor` konfigurációs paraméterei

4.2.3. Cloudflare alagút

A Cloudflare Tunnel egy fordított proxy megoldás, amely lehetővé teszi, hogy a Pi-n futó szolgáltatások nyilvánosan elérhetők legyenek anélkül, hogy a routeren portot kellene nyitni, vagy statikus IP-cím szükséges lenne.[10] A `cloudflared` démon a Pi-ről kifelé irányuló titkosított kapcsolatot épít fel a Cloudflare hálózata felé, így a tűzfal és a NAT nem akadályozza az elérést. Ez különösen fontos a RTSP videóstreamek dashboardra való eljuttatásakor.

A Pi-n ez is systemd szolgáltatásként fut, az induláskor automatikusan csatlakozik. Jelenleg ideiglenes (*quick tunnel*) alagutakat használunk, amelyek ingyenesek, azonban minden újraindításkor új, véletlenszerű URL-t kapnak, és időnként megszakadhatnak. Egy esetleges éles üzembe helyezés esetén célszerű lenne állandó (*named tunnel*) alagútra váltani, amely fix URL-t biztosít és robusztusabb kapcsolatot nyújt.

4.3. Cloud Stack

A rendszer lelke teljes valójában a google cloudban található, mely igen széleskörű eszköztárat biztosított számomra a projekt megvalósításához. Az infrastruktúra felépítéséhez Terraform keretrendszert

használtunk.

4.3.1. Terraform keretrendszer

A terraform egy deklaratív nyelvezetű keretrendszer, mely használatával bármilyen infrastruktúrai elem és erőforrás definiálható emberek számára is értelmezhető nyelven.[11] Ezen megvalósítások, verziózhatóak, disztributálhatóak és bármikor újrafelhasználhatóak. Segítségével az alkalmazás minden életszakasza könnyűszerrel menedzselhető, ugyanitt maga az egész folyamat rendkivül könnyen automatizálható.

4.3.2. Terraform működése

A terraform a különböző cloud platformok erőforrásait a saját Api-jain keresztül menedzseli, a közösségnek hála ma már szinte bármelyik szolgáltató rendszerét használhatjuk a terraformon keresztül. Például AWS, Google Cloud, Kubernetes, Git stb. A terraform működése 3 fázisban történik:

Írás-Write

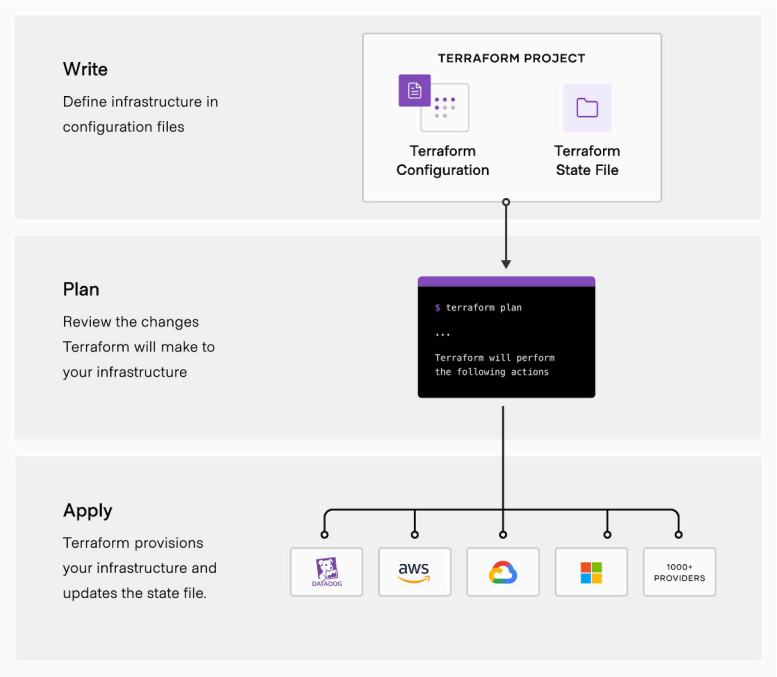
Definiáljuk az erőforrásokat, melyek akár több szolgáltatótól is származhatnak, egy .tf kiterjesztésű fileba.

Tervezés-Plan

A terraform plan parancs kiadása után, létrejön a terv, mely leírja az infrastruktúrát, mi jön létre, mi változik vagy éppen mi szűnik majd meg. Az általunk megadott Konfigurációs fileoknak megfelelően.

Alkalmazás-Apply

Engedélyezés esetén a terraform elvégzi a szükséges műveleteket, tiszteletben tartva az erőforrások közötti függőségi sorrendet. Létrehozza, módosítja vagy törli az érintett erőforrásokat, majd frissíti az állapotfájlt (`terraform.tfstate`), amely a valós infrastruktúra leképezéseként szolgál a jövőbeli futtatásokhoz.



4.3. ábra. Terraform Lépések[1]

4.3.3. Google Cloud Projekt - GCP

A rendszer cloud-oldali erőforrásai egyetlen Google Cloud Projektbe szervezett infrastruktúrán futnak, amelynek egyedi azonosítója `printermonitor-488112`. Az összes felhőszolgáltatás — az Artifact Registry-től a Vertex AI végpontig, a Cloud Function-öktől a Firestore adatbázisig — ezen a projekten belül van definiálva és menedzselve. A Google Cloud Platform (GCP) választásának fő indoka az volt, hogy egységes ökoszisztémát biztosít a gépi tanulási pipeline-hoz: a Vertex AI natívan integrálódik a Pub/Sub üzenetsorral, a Cloud Functions-szel és az Artifact Registry-vel, minimalizálva az integrációs komplexitást.[12] Az erőforrások számlázása és a költségvetési riasztások szintén ezen a projekten belül vannak konfigurálva, lehetővé téve a kiadások szoros nyomon követését.

[1] *Terraform*: https://developer.hashicorp.com/terraform/intro/

4.3.4. Vertex AI végpont

A Vertex AI végpont a detekciós pipeline magja: egy egyedi `judge` Docker konténert futtat, amelyen belül a betanított YOLOv8[5] modell végzi a képkockák valós idejű elemzését. A konténer induláskor betölti a modellt a `MODEL_PATH` által megjelölt `best.pt` fájlból, majd egy HTTP szerveren (`/predict` végpont) vár a bejövő kérésekre.

A szerver kétféle üzenetformátumot kezel: a Vertex AI natív `{"instances": [...]}` formátumát, amelyet a Dispatcher Cloud Function küld, valamint a Pub/Sub push formátumát. Minden beérkező képkockát Base64-ból visszaalakít, OpenCV-vel dekódol, majd a YOLO modellel elvégzi az objektumdetekciót. Az eredmény tartalmazza az észlelt hibaosztályokat (`label`), azok konfidenciaértékeit, valamint a határoló téglalapok koordinátáit mind pixel-, mind normalizált formában. A detekciós eredmény ezt követően publikálásra kerül a `detections-out` Pub/Sub csatornára.

A végpont csak aktív használat közben generál költséget: tétlen (kitelepített, de nem hívott) állapotban nem számlázódik, míg folyamatos terhelés esetén napi körülbelül 37$-os költséggel kell számolni.

4.3.5. Service Accounts

A szervíz felhasználók, olyan egyedi emailcímmel rendelkező entitások, melyek segítségével, különböző applikációk, virtuális gépek, vagy egyéb megvalósítások kommunikálhatnak a Google Api-val, emberi felhasználók helyett, ezzel biztonságos összeköttetést biztosítva a különböző erőforrások között.

sa-frame-extractor

A pi-n fut szerepe, hogy a kinyert képkockák publikálva legyenek a pub/sub csatornára. Jogosultságai, hogy publikál a frames-in csatornára, és aiplatform felhasználó

sa-dispatcher

Az ő szerepete hogy leossza a streameket, és meghívja a VertexAi endpointot. Jogosultságai, hogy aiplatform felhasználó, pub/sub feliratkozó a detections-out csatornán, és aktiválhatja az Alert-managert

szügség esetén

sa-alert-manager

A figyelmeztetés kezelésért felelő, egyaránt az anomáliák észlelésekor, és a költségvetés meghaladásakor. Jogosultságai: `datastore.user`, `firebase.admin`, `eventarc.eventReceiver`, `pubsub.subscriber`, `run.invoker`.

sa-cicd

Mivel az, hogy a terraform infrát mi kézzel szerkesszük nem egy jó praktika, így ehhez egy autómatizált github akciót használunk, ezzel szervíz pont ennek kezeléséért felelős. Jogosultságai: `artifactregistry.writer`, `pubsub.admin`, `editor`, `iam.serviceAccountUser`

Vertex AI agent

A Vertex AI endpoint a `judge` (bíró) Docker konténer futtatásáért felelős. Jogosultságai: `pubsub.publisher` a `detections-out` csatornán.

Pub/Sub agent

A Pub/Sub csatornák menedzselésére szolgál. Jogosultságai: `iam.serviceAccountTokenCreator`.

4.3.6. Pub/Sub Topics

A Google Cloud Pub/Sub egy aszinkron üzenetközvetítő szolgáltatás, amely lehetővé teszi az alkalmazáskomponensek közötti laza csatolású kommunikációt.[13] A rendszer három darab pub/sub csatornát használ, melyek a következők: `frames-in`, mely a még nyers képkockákat továbbítja a pi-től a Vertex AI végpont felé. A `detections-out`, mely a Vertex AI végpont által észlelt anomáliák eredményét továbbítja a dashboard felé, és a `budget-notifications`, mely figyelmeztet minket amennyiben átléptük a kiszabott költségvetést.

4.3.7. Google Cloud Függvények

A rendszer három Cloud Function (Gen2) példányt alkalmaz, amelyek Pub/Sub eseményekre triggerelődnek és kiszolgáltatás nélkül, skálázhatóan futnak.[12] Mindhárom függvény az `sa-alert-manager` illetve `sa-dispatcher` service account jogosultságaival rendelkezik.

Dispatcher

A Dispatcher hidat képez az edge eszköz és a Vertex AI végpont között. A `frames-in` Pub/Sub csatornáról kap üzenetet, amelynek tartalma egy Base64-kódolt JPEG képkocka, a kamera azonosítója, egy szekvenciaszám és egy UTC időbélyeg. A függvény ezt az adatot a Vertex AI REST API-jára továbbítja (`/predict` végpont), OAuth2 Bearer tokennel hitelesítve. Ha a Vertex AI kérés hálózati hiba miatt meghiúsul, a függvény újra dobja a kivételt, így a Pub/Sub automatikusan újrapróbálkozik. A három Cloud Függvény közül ez a leginkább memóriaigényes, mivel képkockák bináris adatát szállítja HTTP-n keresztül.

Alert-Manager

A detekciós pipeline záró állomása. A `detections-out` csatornáról érkező üzenetekből kiszűri azokat a detektált hibákat, amelyek konfidenciaértéke eléri a beállított küszöböt (alapértelmezetten 35%). Az átmenő detektálásokat a Firestore `alerts` kollekcióba írja, rögzítve a kamera azonosítóját, az észlelt hibákat, az időbélyeget és a szekvenciaszámot.

E-mail értesítést kizárólag magas súlyosságú hibaosztályok esetén küld: `spagetti`, `not_sticking`, `layer_shift` és `warping`. Az ismétlődő riasztások elkerülése érdekében kameránként 60 másodperces visszatartási időt (*cooldown*) alkalmaz, amelyet a Firestore `alert_cooldowns` kollekcióban tárol. Az e-mail HTML formátumú, tartalmazza az észlelt hibák táblázatát és egy közvetlen linket a dashboardra.

Budget Notifier

E függvény célja, hogy megakadályozza a felhőköltségek elszabadulását. A `budget-notifications` Pub/Sub csatornán hallgatózik, amelyre a GCP saját költségvetési riasztó rendszere publikál, ha az

előre beállított kiadási küszöb teljesül. A függvény az Alert-Managerrel közös kódbázisban van megvalósítva, és szintén cooldown mechanizmust alkalmaz. Az eseményt a Firestore `budget_alerts` kollekcióba is naplózza, majd HTML e-mailben értesíti a felhasználót a felhasznált összegről és a küszöbértékről.

4.3.8. Firestore Collections

Az adatbáziskezelő rendszereknek két nagy fajtáját különböztetünk meg, a relációs adatbázisokat, és a nem relációs avagy NoSQL (Not only SQL) adatbázisokat. A fő különbség , hogy a reláciiós adatbázisokban az adatok táblákba rendezettek, és köztük jól meghatározott kapcsolatok vannak. Viszont a NoSql adatbázisokban az adatok úgynevezett dokumentumokban vannak eltárolva, melyek szerkezete sokkal rugalmasabb és nem igényel előre meghatározott sémát. Mivel a projekt főként json formátumú adatokkal dolgozik, így a NoSQL adatbázisok sokkal jobban megfelelnek a céljainknak, és a Firestore egy igen népszerű és megbízható választás ezen a téren.[14]

A rendszer összesen három Firestore kollekciót használ:

Az `alerts` kollekció tárolja a detektált anomáliák bejegyzéseit. Mezői: `camera_id` (melyik kamera észlelte), `detections[]` (a hibák tömbje: név + konfidencia + bounding box), `timestamp` (mikor történt a detekció), `seq` (a képkocka szekvenciaszáma, amelyet a `frame-extractor` biztosít) és `created_at` (mikor került be a bejegyzés). A dashboardon való hatékony keresést egy kompozit index segíti: `camera_id ASC + timestamp DESC`, azaz kamera szerint növekvő, időbélyeg szerint csökkenő sorrendben.

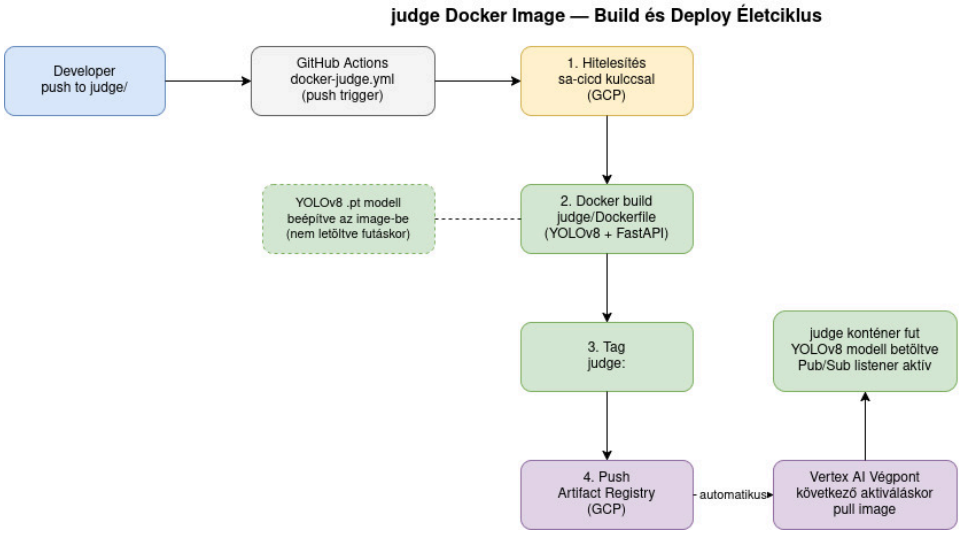
Az `alert_cooldowns` kollekció az e-mail értesítések visszatartási idejét kezeli kameránként, megakadályozva a riasztás-özönt rövid időn belül ismétlődő anomáliák esetén.

A `budget_alerts` kollekció a GCP költségvetési riasztásokat naplózza, beleértve a felhasznált összeget, a beállított küszöböt és az esemény időbélyegét.

4.3.9. Konténer képek

A Vertex AI végponton futó detekciós logika egy egyedi Docker konténer képbe (image) van csomagolva, melynek neve `judge`. Ez az image egy Python scriptet tartalmaz, amely induláskor betölti a

betanított YOLOv8 modellt, majd várakozik a bejövő Pub/Sub üzenetekre. Egy üzenet érkezésekor kicsomagolja a base64-kódolt JPEG képkockát, elvégzi rajta az objektumdetekciót, majd az eredményt — a detektált hibaosztályokat és azok konfidenciaértékeit — visszapublikálja a `detections-out` Pub/Sub csatornára.[15] Az image tárolása a Google Artifact Registry-ben történik, verziózva a commit SHA alapján. Minden módosítás esetén a `docker-judge.yml` GitHub Action automatikusan újrabuildi és feltölti a friss image-t, ahonnan a Vertex AI végpont azt lehúzza és futtatja.



4.4. ábra. A `judge` konténer image életciklusa

4.4. Dashboard

A rendszer felhasználói felülete egy egyoldalas webalkalmazás (Single Page Application), melyet a Firebase Hosting platform szolgál ki, így külön szerver üzemeltetése nélkül érhető el a böngészőből.[16] Az alkalmazás két fő funkciót lát el párhuzamosan.

Egyrészt valós időben jeleníti meg a három kamera videofolyamát WebRTC technológia segítségével, a MediaMTX szerveren és a Cloudflare alagúton keresztül érkező RTSP streamekre támaszkodva. Másrészt a Firebase Firestore adatbázishoz való közvetlen, valós idejű kapcsolaton keresztül (`onSnapshot` listener) azonnal megjelenik minden újonnan detektált anomália, azok neve és konfidenciaértéke, valamint az észlelés időbélyege és a forrás kamera azonosítója.

A dashboard vizuálisan is visszajelzést ad a rendszer állapotáról: amennyiben magas prioritású

anomália érkezik, a felület piros kerettel jelenik meg (CSS `alerting` osztály aktiválásával). Hasonlóan, a költségvetési riasztás esetén sárga kerettel jelez (`budget-alert` osztály), így a felhasználó azonnal értesül a túllépésről anélkül, hogy az e-mailjét kellene ellenőriznie.

[ÁBRA: Dashboard normál nézet screenshot]

4.5. ábra. A dashboard normál működési állapotban

[ÁBRA: Dashboard riasztási állapot screenshot (piros keret)]

4.6. ábra. A dashboard anomália detektálás esetén

4.5. Github Action Workflows

A projekt folyamatos integrációját és automatizált kitelepítését (CI/CD) két GitHub Action workflow biztosítja, melyek a megfelelő kódváltozásra automatikusan lefutnak, csökkentve a manuális beavatkozás szükségességét és az emberi hibák esélyét.[17]

4.5.1. Terraform lépések automatizálása

Ez a workflow a cloud infrastruktúra változtatásait kezeli automatikusan. Pull request esetén csak ellenőrző lépések futnak le, hogy a változtatások biztonságosak és helyesek legyenek merge előtt.

Merge eseményre — a `main` ágra — a workflow elvégzi a tényleges infrastruktúra módosítást a GCP-n.

Konfigurációs ellenőrzés-Chekov

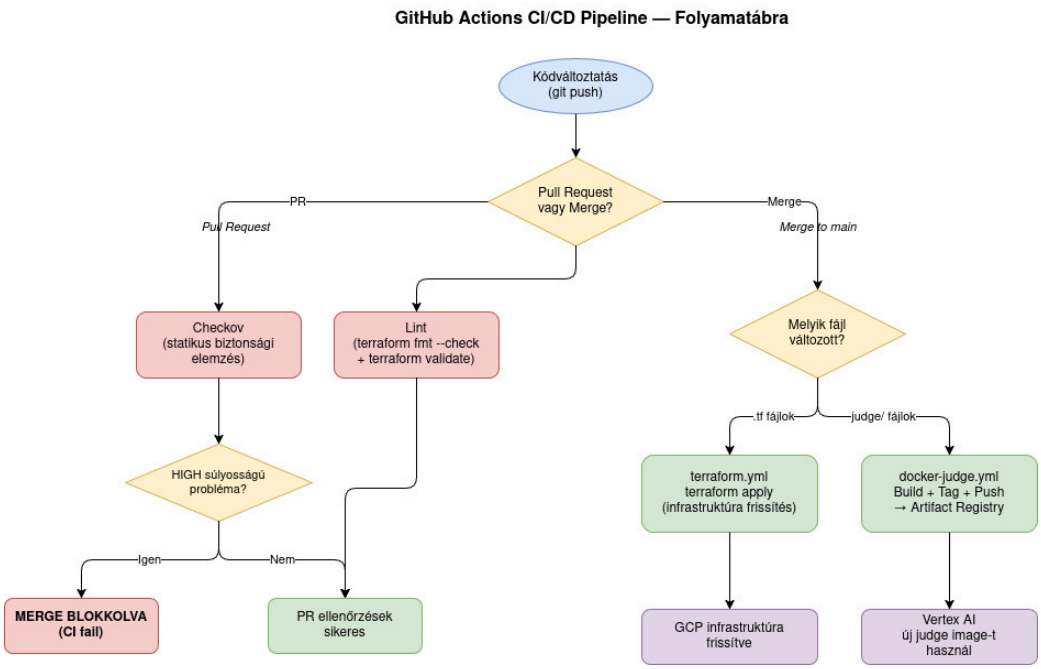
A Checkov egy nyílt forráskódú statikus elemző eszköz, amely Terraform konfigurációkat vizsgál ismert biztonsági problémák és rossz praktikák szempontjából.[18] Futtatása pull request esetén kötelező lépés: amennyiben magas súlyosságú problémát talál — például nyilvánosan elérhető storage bucket vagy nem megfelelő IAM konfiguráció —, a workflow megakadályozza a merge-t. Ez biztosítja, hogy a fejlesztés során se kerülhessen sérülékeny infrastruktúra a produkcióba.

Formázás és erőforráshasználat ellenőrzés-Lint

A linting lépés két dolgot ellenőriz. A `terraform fmt -check` parancs megvizsgálja, hogy a `.tf` fájlok formázása megfelel-e a Terraform hivatalos stílusirányelveknek. A `terraform validate` parancs pedig szintaktikailag és szemantikailag ellenőrzi a konfigurációt, kiszűrve a nem létező erőforrásokra való hivatkozásokat vagy érvénytelen argumentumokat. Ezzel a lépéssel a triviális hibák már a review fázisban kiszűrhetők.

4.5.2. Docker konténer felépítésének automatizálása

Ez a workflow a `judge` Docker image buildjeléért és az Artifact Registry-be való automatikus feltöltéséért felelős. Minden olyan push eseményre lefut, amely a `judge/` könyvtárat érinti — így csak releváns változtatás esetén indul el, nem feleslegesen. A workflow lépései sorban: hitelesítés az `sa-cicd` service account kulcsával, az image buildje, ellátása a commit SHA-val mint egyedi taggel, majd feltöltése a Google Artifact Registry-be. Ezt követően a Vertex AI végpont a következő aktiváláskor automatikusan az új image-t fogja használni.



4.7. ábra. A CI/CD pipeline lépései

Összefoglalás

A fejezet bemutatta a rendszer teljes hardver- és szoftverkörnyezetét. Az edge oldalon egy Raspberry Pi 4B gondoskodik a kameraképek rögzítéséről és valós idejű továbbításáról, a felhő oldalon pedig a Google Cloud Platform szolgáltatásai — Pub/Sub, Cloud Functions, Vertex AI, Firestore és Firebase Hosting — alkotják azt a pipeline-t, amely a nyers képkockáktól az anomáliadetekción át a felhasználói értesítésig minden lépést lefed. Az infrastruktúra Terraformmal van definiálva és verziókezelve, a CI/CD pipeline pedig biztosítja, hogy minden kódváltozás automatikusan, emberi beavatkozás nélkül jusson el az éles környezetbe.

5. fejezet

Adatgyűjtés és előfeldolgozás

5.1. Cím 1

6. fejezet

Modell betanítása és validálása

6.1. Cím 1

Eredmények leírása

7. fejezet

Tesztelés és kiértékelés

7.1. Összefoglalás

8. fejezet

Rendszerintegráció és alkalmazás

9. fejezet

Összefoglalás és jövőbeli irányok

10. fejezet

Mellékletek

Irodalomjegyzék

[1] Voxel Matters, „The Rise of 3D Printer Farms.” https://www.voxelmatters.com/the-rise-of-3d-printer-farms/, 2023. Hozzáférés: 2026.

[2] Obico, „Obico: Open-Source AI-Powered 3D Printing Failure Detection.” https://www.obico.io/, 2021. Hozzáférés: 2026.

[3] S. Ren, K. He, R. Girshick, and J. Sun, „Faster R-CNN: Towards real-time object detection with region proposal networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 28, 2015.

[4] G. Jocher, „YOLOv5 by Ultralytics.” https://github.com/ultralytics/yolov5, 2020. Hozzáférés: 2026.

[5] Ultralytics, „YOLOv8: A new state-of-the-art computer vision model.” https://github.com/ultralytics/ultralytics, 2023. Hozzáférés: 2026.

[6] Intel Corporation, „CVAT: Computer Vision Annotation Tool.” https://github.com/cvat-ai/cvat, 2019. Hozzáférés: 2026.

[7] Raspberry Pi Ltd., „Raspberry Pi 4 Model B.” https://www.raspberrypi.com/products/raspberry-pi-4-model-b/, 2024. Hozzáférés: 2026.

[8] bluenviron, „MediaMTX: Ready-to-use SRT / WebRTC / RTSP / RTMP / LL-HLS media server.” https://github.com/bluenviron/mediamtx, 2024. Hozzáférés: 2026.

[9] OpenCV Team, „OpenCV: Open Source Computer Vision Library.” https://opencv.org/, 2024. Hozzáférés: 2026.

[10] Cloudflare Inc., „Cloudflare Tunnel Documentation.” https://developers.cloudflare.com/cloudflare-one/connections/connect-networks/, 2024. Hozzáférés: 2026.

[11] HashiCorp Inc., „Terraform: Infrastructure as Code.” https://developer.hashicorp.com/terraform/intro, 2024. Hozzáférés: 2026.

[12] Google LLC, „Google Cloud Platform Documentation.” https://cloud.google.com/docs, 2024. Hozzáférés: 2026.

[13] Google LLC, „Google Cloud Pub/Sub Documentation.” https://cloud.google.com/pubsub/docs, 2024. Hozzáférés: 2026.

[14] Google LLC, „Cloud Firestore Documentation.” https://firebase.google.com/docs/firestore, 2024. Hozzáférés: 2026.

[15] Google LLC, „Vertex AI: Use a custom container for prediction.” https://cloud.google.com/vertex-ai/docs/predictions/use-custom-container, 2024. Hozzáférés: 2026.

[16] Google LLC, „Firebase Hosting Documentation.” https://firebase.google.com/docs/hosting, 2024. Hozzáférés: 2026.

[17] GitHub Inc., „GitHub Actions Documentation.” https://docs.github.com/en/actions, 2024. Hozzáférés: 2026.

[18] Bridgecrew Inc., „Checkov: Static code analysis tool for infrastructure as code.” https://www.checkov.io/, 2024. Hozzáférés: 2026.

A. függelék

Függelék

A.1. Alfejezet

A.1.1. Cím

Alcím